

Cursos

Academia Código Norte · la primera relación entre tablas

Duración	Qué vas a aprender	Necesitas
1 sesión (~1 h)	Clave ajena (FOREIGN KEY), CHECK y restricciones con nombre	Haber terminado la práctica 1

Dónde estamos

Aparece el **CURSO**. Cada curso lo da **un** profesor, y un profesor puede dar **varios** cursos: es una relación **1:N**. En la tabla, eso se traduce en una columna `id_profesor` dentro de `curso` que apunta al profesor.



En naranja, lo nuevo de esta práctica.

Lo nuevo de hoy

- **Clave ajena (FOREIGN KEY):** una columna que apunta a la clave primaria de otra tabla. Con ella, la base de datos **no te deja** poner un curso con un profesor que no existe.
- **ON DELETE RESTRICT:** mientras un profesor tenga cursos, no se puede borrar.
- **CHECK:** una condición que el dato tiene que cumplir. Por ejemplo, que las horas sean mayores que 0.
- **CONSTRAINT nombre:** le ponemos nombre a una regla para reconocerla cuando salte un error.

La tabla CURSO

Este es el código completo. Léelo con calma: la mayoría ya lo conoces de la práctica 1, y lo nuevo te lo explicamos debajo.

```
CREATE TABLE curso (  
  id_curso      INT          PRIMARY KEY,  
  nombre        VARCHAR(80)  NOT NULL UNIQUE,  
  nivel         VARCHAR(12)  NOT NULL CHECK (nivel IN ('Básico', 'Intermedio', 'Avanzado')),  
  horas         INT          NOT NULL CHECK (horas > 0),  
  precio        NUMERIC(7,2) NOT NULL CHECK (precio >= 0),  
  fecha_inicio  DATE         NOT NULL,  
  fecha_fin     DATE         NOT NULL,  
  id_profesor   INT          NOT NULL,  
  CONSTRAINT ck_curso_fechas CHECK (fecha_fin > fecha_inicio),  
  CONSTRAINT fk_curso_profesor FOREIGN KEY (id_profesor)  
    REFERENCES profesor (id_profesor) ON DELETE RESTRICT  
);
```

Qué guarda cada columna:

Columna	Tipo	Reglas	Qué guarda
id_curso	INT	PRIMARY KEY	Número del curso
nombre	VARCHAR(80)	NOT NULL UNIQUE	Nombre del curso
nivel	VARCHAR(12)	NOT NULL CHECK (nivel IN ('Básico', 'Intermedio', 'Avanzado'))	Solo puede ser uno de esos tres
horas	INT	NOT NULL CHECK (horas > 0)	Horas totales
precio	NUMERIC(7,2)	NOT NULL CHECK (precio >= 0)	Precio en euros
fecha_inicio	DATE	NOT NULL	Primer día
fecha_fin	DATE	NOT NULL	Último día
id_profesor	INT	NOT NULL	Quién lo imparte (clave ajena)

Las dos últimas líneas no son columnas: son **reglas de tabla**. Van al final, separadas por comas como las columnas:

Esta regla...	...quiere decir
ck_curso_fechas	El curso no puede acabar antes de empezar. Va al final porque usa dos columnas.
fk_curso_profesor	La columna id_profesor de curso apunta a id_profesor de la tabla profesor. Si el profesor tiene cursos, no se puede borrar.

Pista

id_profesor en **curso** es **INT**, igual que **id_profesor** en **profesor**. Una clave ajena tiene que ser **del mismo tipo** que la columna a la que apunta.

Paso a paso

Hoy tendrás **3 consultas guardadas** en el SQL Editor, cada una con su nombre (igual que en la práctica 1):

Nombre de la consulta	Qué tiene dentro
P2 · 1 · Tabla curso	El CREATE TABLE curso
P2 · 2 · Datos de prueba	Los INSERT de cursos

Nombre de la consulta	Qué tiene dentro
P2 · 3 · Intenta romperla	Las cuatro pruebas que deben fallar

1. Crea cada consulta **en el orden de la tabla**: **New query**, pega, **Run** y, si sale «Success», renómbrala (tres puntos → **Rename query**).
2. En el **Table Editor**, abre **curso**: la columna **id_profesor** tiene un icono de enlace. Es la clave ajena.
3. Ejecuta las pruebas de **Intenta romperla** una a una (selecciona cada una y pulsa **Run**).

Datos de prueba

```
INSERT INTO curso
(id_curso, nombre, nivel, horas, precio, fecha_inicio, fecha_fin, id_profesor)
VALUES
(1, 'Python desde cero', 'Básico', 40, 180.00, '2026-10-05', '2026-11-27', 1),
(2, 'SQL para el mundo real', 'Intermedio', 30, 150.00, '2026-10-13', '2026-11-20', 2),
(3, 'HTML y CSS sin miedo', 'Básico', 25, 120.00, '2026-10-06', '2026-11-05', 3),
(4, 'Proyecto web con Next.js', 'Avanzado', 50, 260.00, '2027-01-11', '2027-03-19', 3);
```

Comprueba

curso tiene 4 filas. El último número de cada fila es el profesor: 1 es Andrés, 2 es Irene y 3 es Hugo.

Intenta romperla

Ejecuta estas sentencias **una a una**. Las cuatro deben dar error:

#	Sentencia	Debe fallar porque...
1	INSERT INTO curso (id_curso, nombre, nivel, horas, precio, fecha_inicio, fecha_fin, id_profesor) VALUES (5, 'Fantasma', 'Básico', 10, 50, '2026-11-01', '2026-12-01', 99);	No existe el profesor 99 (fk_curso_profesor)
2	INSERT INTO curso (id_curso, nombre, nivel, horas, precio, fecha_inicio, fecha_fin, id_profesor) VALUES (5, 'Curso top', 'Experto', 10, 50, '2026-11-01', '2026-12-01', 1);	'Experto' no es un nivel válido (CHECK)
3	INSERT INTO curso (id_curso, nombre, nivel, horas, precio, fecha_inicio, fecha_fin, id_profesor) VALUES (5, 'Curso raro', 'Básico', 10, 50, '2026-12-01', '2026-11-01', 1);	Acaba antes de empezar (ck_curso_fechas)
4	DELETE FROM profesor WHERE id_profesor = 1;	Andrés tiene cursos (ON DELETE RESTRICT)

Piensa

Fíjate en el mensaje de error de la prueba 3: sale el nombre **ck_curso_fechas**. ¿Por qué es útil ponerles nombre a las reglas?

¿Por qué la columna **id_profesor** está en **curso** y no al revés, una columna **id_curso** en **profesor**?

¿Te has equivocado? Empieza de nuevo

Ejecuta esto y vuelve a lanzar tu código de esta práctica desde el principio. Solo borra lo de **esta** práctica: lo de las anteriores no se toca.

```
DROP TABLE IF EXISTS curso CASCADE;
```

Qué entregas

Captura de la lista de consultas con las de la práctica 2 (P2 · ...) y captura del error de la prueba 4.

Anclaje curricular · RD 405/2023 · Módulo 0484 · RA2 · **criterios b, c, d, e y h**